Fake Gang Detection — OpenEnv RL Environment

An OpenEnv-compatible reinforcement learning environment where an LLM detective must identify all 10 members of a coordinated fake Instagram account gang hidden inside a synthetic social network. The agent learns via **Reflexion** and a **dynamic hybrid rule/LLM policy** — no gradient updates, no fine-tuning.

Table of Contents

---

1. What This Is

This is an **OpenEnv hackathon** submission. OpenEnv is a framework for building reinforcement learning environments with a standard microservice interface (`/reset`, `/step`, `/state`) so that any agent implementation can plug in.

The task: A social network contains 1000 fake accounts organised into a single "gang" of 10. The gang behaves in a coordinated way — same posting hour, same IP subnet, stolen celebrity photos, copy-paste bios. The agent must find all 10 by navigating a limited step budget, inspecting accounts, and flagging suspects.

What makes this non-trivial:

- The network is large (50–1000 accounts depending on difficulty).
- Fake accounts are mixed with innocent high-signal "decoy" accounts.
- In hard mode, the gang actively evades — dropping intra-gang follows, renaming profiles — while the agent is mid-investigation.
- The agent cannot see the full network upfront: it must explore via INSPECT and INVESTIGATE_NETWORK actions, spending steps to reveal information.

What makes the learning novel:

- The LLM (Qwen3-80B via AWS Bedrock) cannot be fine-tuned — it is a black-box API.
- The agent learns via **Reflexion**: post-episode lessons are written back into memory and injected into every future prompt.
- A **dynamic hybrid policy** (α-weighted) blends the LLM with a deterministic rule engine, with the blend weight α updating based on recent win rate. Rules dominate early; the LLM takes over as it proves itself.

---

2. Repository Layout

```
fake_gang_env/
│
├── models.py                  # All Pydantic types: Action, Observation,
State, Profile
├── bedrock_model.py           # AWS Bedrock client — invoke_qwen()
├── client.py                  # HTTP client for talking to the running
server
├── inference.py               # Submission: rule-based baseline runner +
HTTP client mode
├── validate.py                # Submission: pre-submission validator (24
checks)
```

```
├── train.py                    # Main training loop (curriculum + hybrid
policy)
├── run.sh                      # Docker entrypoint: episodes → server →
training
├── requirements.txt            # Python dependencies
│
├── server/
│   ├── app.py                  # FastAPI server: /reset /step /state /health
/tasks /grader /baseline
│   ├── environment.py          # Core RL environment — FakeGangEnvironment
class
│   ├── generator.py            # Synthetic episode generator (50 per task ×
3 tasks = 150 files)
│   ├── scoring.py              # Pure-math risk formula engine (stateless
functions)
│   ├── Dockerfile              # Offline pip install via pre-downloaded
wheels
│   └── .dockerignore           # Excludes episodes/, memory/, runs/ from
build context
│
├── agent/
│   ├── policy.py               # LLM policy: formats obs → calls Qwen →
parses <action> tag
│   ├── hybrid_policy.py        # Hybrid policy: blends rules + LLM via
dynamic α
│   ├── memory.py               # Disk-backed memory: reflections,
trajectories, win history, α
│   └── reflection.py           # Post-episode reflection generator (also
calls Qwen)
│
├── episodes/                   # 150 pre-generated JSON episode files
(excluded from Docker build)
├── memory/                     # Docker volume: reflections, trajectories, α
values (persists)
└── runs/                       # Docker volume: per-episode metrics JSONL
(persists)
```

---

3. The Problem: How Fake Detection Actually Works

A real-world fake account detector does **not** read post content. Content is expensive to process, multilingual, and easily spoofed. Instead, detection relies on three categories of signals that are computed from metadata:

3.1 Node Signals (per-account features)

These are pre-computed by a content pipeline before the agent ever sees the account:

Feature	What it measures	Fake range	Real range
`photo_reuse_score`	Fraction of posts using stolen celebrity photos (pHash fingerprint matching)	0.30–0.95	0.00–0.15
`bio_template_score`	Cosine similarity of bio text to known fake-account bio templates	0.20–0.90	0.00–0.12
`comment_repeat_score`	Fraction of comments that are copy-pasted across accounts (spam pattern)	0.60–0.90	0.00–0.08

The agent receives these as numbers (not the raw content). The content pipeline runs offline; the agent operates on numeric features only.

3.2 Behavioral Signals (temporal + device)

Feature	What it measures	Fake pattern
`avg_post_hour`	Mean hour of day for all posts (0–23)	Gang members cluster within ±0.5 hours (coordinated scheduling)
`account_age_days`	Days since account creation	Gang created within same week (base_age ± 7 days)
`shared_ip_count`	How many other accounts share the same IP subnet	9 (all 10 gang members share one IP)
`ip_cluster_id`	Identifier for the IP subnet	All fakes:`ip_gang_{seed}`

3.3 Structural / Graph Signals (derived at INSPECT time)

These cannot be pre-computed because they depend on **which accounts the agent has already flagged** — they change dynamically as the investigation progresses:

Feature	What it measures	Fake pattern
`mutual_follow_rate`	Fraction of this account's follows that also follow back	Gang mutually follow each other heavily (0.6–0.9)
`flagged_neighbor_count`	How many of this account's follows are currently flagged	High → embedded in the cluster you're tracking
`avg_neighbor_photo_reuse`	Mean photo_reuse_score of already-inspected follows	Gang members cluster; inspecting one reveals others
`post_hour_cluster_score`	How closely posting hour matches the flagged accounts' mean	High → same operation schedule
`suspicious_mutual_ratio`	Fraction of suspicious follows that also follow this account back	Used in hub legitimacy computation

Why graph signals are the most powerful: A gang member always has high `mutual_follow_rate` and grows `flagged_neighbor_count` as the investigation proceeds. A celebrity who is *followed by* fakes but does not *follow them back* will have low `mutual_follow_rate` — this is how the hub legitimacy score protects against false positives.

---

4. Synthetic Data Generation

File: `server/generator.py`

Episodes are generated deterministically by seed. 150 episodes are pre-generated (50 per task) and cached as JSON files in `episodes/`.

4.1 Network Composition

Task	Network size	Gang	Decoys	Real	Max steps
easy	50	10	0	40	30
medium	200	10	20	170	50
hard	1000	10	50	940	80

Gang accounts: All 10 share:

- A `base_age` (same creation week): `account_age_days = base_age + randint(0, 7)`
- Tightly clustered `avg_post_hour`: sampled from `gauss(14.0, std)` where std=0.5/1.5/2.5 by task
- High `photo_reuse_score` and `bio_template_score` (ranges defined per task)
- `comment_repeat_score` in [0.60, 0.90]
- `ip_cluster_id = "ip_gang_{seed}"` and `shared_ip_count = 9`
- Dense intra-gang follow edges (density 0.60–0.80 by task)

Real accounts: Drawn from log-normal distributions mimicking real Instagram stats:

- `follower_count`: log-normal(μ=6.0, σ=1.5), clipped to [10, 100,000]
- `comment_repeat_score`: Beta(1, 20) ≈ mostly 0.00–0.08
- Each has a unique `ip_cluster_id`

Decoy accounts (medium/hard only): Real accounts with elevated fraud scores:

- `photo_reuse_score` in [0.20, 0.40], `bio_template_score` in [0.20, 0.40]
- `comment_repeat_score` in [0.10, 0.30]
- They look suspicious but are NOT gang members — they penalise reckless flagging.

Celebrity accounts (2 per episode): Very high follower counts, very low fake scores:

- `follower_count` in [100,000, 5,000,000]
- `photo_reuse_score` in [0.00, 0.05], `bio_template_score` in [0.00, 0.05]
- High `hub_legitimacy_score` → the formula protects them from false positives.

Zero-edge isolates (2 per episode): Real accounts with `follower_count=0`, `following_count=0`, no graph edges. They test whether the agent wastes steps on disconnected nodes.

4.2 Edge Generation

Intra-gang edges are added first with high density:

```
for g in gang_ids:
    for h in gang_ids:
        if g != h and random() < density:
            g follows h
```

Real and decoy accounts get sparse preferential-attachment edges: each follows 5–50 random other accounts. This creates a realistic social graph where gang members are much more tightly interconnected than real users.

4.3 Episode JSON Schema


```
{
  "episode_id": "uuid4",
  "task": "easy",
  "seed": 0,
  "max_steps": 30,
  "win_recall": 0.8,
  "win_precision": 0.7,
  "starting_visible": ["acc_0012", "acc_0037", ...],
  "gang_member_ids": ["acc_0003", "acc_0017", ...],
  "decoy_ids": [],
  "celeb_ids": ["acc_0048", "acc_0049"],
  "zero_edge_ids": ["acc_0046", "acc_0047"],
  "network": {
    "accounts": [
      {
        "id": "acc_0003",
        "is_fake": true,
        "gang_id": "gang_A",
        "features": {
          "follower_count": 3421,
          "following_count": 847,
          "post_count": 214,
          "avg_post_hour": 14.23,
          "photo_reuse_score": 0.8712,
          "bio_template_score": 0.7403,
          "account_age_days": 67,
          "comment_repeat_score": 0.7831,
          "ip_cluster_id": "ip_gang_0",
          "shared_ip_count": 9,
          "name_change_count": 0
        },
        "true_edges": {
```

```
            "follows": ["acc_0017", "acc_0029", ...],
            "followed_by": ["acc_0017", "acc_0008", ...]
          }
        }
      ]
    },
    "evasion_schedule": []
  }
```

---

5. Data Model — Every Field Explained

File: `models.py`

5.1 ActionType (enum)

Value	Cost	Effect
`inspect`	1 step	Reveals full `AccountProfile` + follow list; adds neighbors to `visible_account_ids`
`investigate_network`	2 steps	Expands 2 hops from account; only reveals account IDs (no profiles)
`flag`	0 steps	Marks account as gang member; triggers SUSPECT cascade to visible neighbors
`unflag`	0 steps	Removes flag; clears CONFIRMED_FAKE status
`submit`	0 steps	Ends episode; triggers scoring

5.2 AccountStatus (enum)

```
NORMAL          → no signal or formula risk < 0.35
SUSPECT         → auto-elevated: a flagged neighbor follows this account
CONFIRMED_FAKE  → agent explicitly flagged this account
```

Transitions are one-directional except UNFLAG which clears CONFIRMED_FAKE. SUSPECT is set automatically — the agent never sets it manually.

5.3 AccountProfile — all 22 fields

```
account_id: str               # "acc_0042"

# Raw counts
follower_count: int           # followers (fakes: 1k-8k, celebs: 100k-5M)
following_count: int          # accounts followed (fakes: 200-2000)
post_count: int               # total posts (fakes: 50-500)

# Temporal
avg_post_hour: float          # mean posting hour 0-23 (gang: tightly
clustered)
account_age_days: int         # days since creation (gang: same week, within
7 days)

# Content pipeline scores (pre-computed, 0-1)
photo_reuse_score: float      # pHash stolen-photo detection
bio_template_score: float     # cosine similarity to fake bio templates
comment_repeat_score: float   # copy-paste spam comment fraction

# IP / device
shared_ip_count: int          # accounts sharing same IP subnet (gang: 9)

# Graph features (computed live at INSPECT time)
mutual_follow_rate: float     # fraction of follows that follow back (fakes:
0.6-0.9)
flagged_neighbor_count: int   # follows already flagged (grows as
investigation progresses)
avg_neighbor_photo_reuse: float   # mean photo_reuse of inspected follows
inspected_neighbor_count: int     # denominator for flagged_neighbor_ratio
post_hour_cluster_score: float    # hour alignment to flagged cluster mean
(0-1)
suspicious_mutual_ratio: float    # mutual rate among suspicious follows
(for hub legitimacy)

# Risk breakdown (computed via scoring.py at INSPECT time)
fake_risk_score: float        # composite 0-1 (main decision signal)
node_risk: float              # from photo_reuse + bio_template
behavior_risk: float          # from account_age + post_hour_cluster
graph_risk: float             # from flagged_neighbor_ratio + mutual +
avg_neighbor
hub_legitimacy_score: float   # celebrity/hub discount

# Evasion tracking
name_change_count: int        # incremented by hard-mode evasion events

# Status
status: AccountStatus         # NORMAL / SUSPECT / CONFIRMED_FAKE
visible_follows: List[str]    # follow list revealed by INSPECT
```

5.4 FakeGangObservation — what the agent sees each step

```
done: bool                    # episode over?
reward: Optional[float]       # only set on terminal step
visible_accounts: List[AccountProfile]   # fully profiled (inspected)
accounts
visible_account_ids: List[str]           # all known account IDs (profiled +
seen)
flagged_ids: List[str]        # currently flagged by agent
inspected_ids: List[str]      # accounts with full profiles revealed
suspect_ids: List[str]        # auto-elevated SUSPECT accounts (uninspected
cascade)
graph_edges: Dict[str, List[str]]        # follow lists for inspected
accounts
steps_remaining: int          # budget left
evasion_triggered: bool       # was evasion active this episode?
evasion_count: int            # how many evasion events have fired
task: str                     # "easy" / "medium" / "hard"
message: str                  # human-readable result / status message
```

---

6. The RL Environment

File: `server/environment.py`

6.1 Episode Lifecycle

```
reset(task, seed)
    └── loads JSON episode file (or generates on the fly)
    └── initialises _visible_ids with starting_visible accounts
    └── returns initial observation (no profiles yet)

step(action)  [called repeatedly]
    └── INSPECT  → _do_inspect()  → reveals profile + neighbors
    └── FLAG     → _do_flag()     → cascades SUSPECT to visible neighbors
    └── UNFLAG   → _do_unflag()   → clears status
    └── INVESTIGATE_NETWORK → _do_investigate() → reveals 2-hop IDs
    └── SUBMIT   → _do_submit()   → scores and ends episode

    If step_count >= max_steps → forced submit (penalty -2.0)
```

6.2 Action Mechanics in Detail

INSPECT (1 step):

1. Adds account to `_inspected`
2. Calls `_build_profile(acc_id)` — computes all 22 features dynamically
3. Adds all accounts this account follows to `_visible_ids`
4. Returns updated observation

INVESTIGATE_NETWORK (2 steps):

1. Adds account to `_inspected` (counts it as seen)
2. Reveals 1-hop neighbors AND their 1-hop neighbors (2-hop total)
3. Adds all new account IDs to `_visible_ids` (no full profiles — IDs only)
4. Cost: 2 steps, -0.02 score. Returns count of newly discovered IDs.

FLAG (free):

1. Adds account to `_flagged`
2. Sets `_account_statuses[acc_id] = "confirmed_fake"`
3. **CASCADE:** For every neighbor in `_live_edges[acc_id]`:
   - If the neighbor is in `_visible_ids` AND currently `"normal"`:
   - Set `_account_statuses[neighbor] = "suspect"`
4. Refreshes all already-inspected accounts that follow `acc_id` (their `flagged_neighbor_count` just increased, so risk scores change)

SUBMIT: Computes final scores (see §6.3).

6.3 Reward Function

```
tp = len(gang_ids ∩ flagged_ids)     # true positives
fp = len(flagged_ids - gang_ids)     # false positives
fn = len(gang_ids - flagged_ids)     # false negatives

base_reward = tp×1.0 − fp×0.5 − fn×0.3

Win condition (task-dependent thresholds):
  easy/medium:  recall ≥ 0.8 AND precision ≥ 0.7
  hard:         recall ≥ 0.9 AND precision ≥ 0.8

If WIN:
  +5.0          # full win bonus
  +3.0          # if perfect recall (tp == len(gang_ids))
```

```
elif recall ≥ win_recall:
  +2.0          # partial win (high recall, low precision)

Efficiency bonus (if SUBMIT called voluntarily with ≥50% steps remaining):
  +1.0

Hard mode evasion penalty:
  −1.0 × evasion_count

Forced submit (ran out of steps):
  −2.0

Final score = base_reward + all bonuses/penalties
```

Example: Easy task, found 9/10 gang members, flagged 2 innocent accounts, 30 steps used, submitted voluntarily with 5 steps left (< 50%):

```
tp=9, fp=2, fn=1
base = 9×1.0 − 2×0.5 − 1×0.3 = 9 − 1 − 0.3 = 7.7
recall = 9/10 = 0.90 ≥ 0.8 ✓  precision = 9/11 = 0.82 ≥ 0.7 ✓
+5.0 win bonus
0 efficiency bonus (steps_left=5 < 30×0.5=15)
total = 7.7 + 5.0 = 12.7
```

6.4 Evasion (hard mode)

The `evasion_schedule` in each episode defines trigger points. When `step_count >= event["step"]` and the event hasn't fired yet:

`unfollow_intragang`: A fraction (`drop_rate=0.3`) of intra-gang edges are randomly removed from `_live_edges`. This destroys the graph signal mid-investigation. The agent sees `mutual_follow_rate` and `flagged_neighbor_count` drop on re-inspection. Hard mode fires this 4 times (steps 15, 30, 45, 60).

`rename_count`: A random subset of gang members get `name_change_count += 1`. This is a visual signal — the agent should notice accounts that have changed their name multiple times.

---

7. Risk Scoring Mathematics

File: `server/scoring.py`

All five functions are **stateless and deterministic** — no side effects, no global state. They are called inside `_build_profile()` every time an account is inspected or a neighbor is re-profiled after a FLAG.

7.1 Node Risk

Captures content-based fakeness signals:

```
node_risk = 0.60 × photo_reuse_score + 0.40 × bio_template_score
```

Photo reuse gets 60% weight because it is harder to spoof (requires actual pHash fingerprint matching against a celebrity photo database).

Example: Gang member with `photo_reuse=0.87`, `bio_template=0.74`:

```
node_risk = 0.60 × 0.87 + 0.40 × 0.74 = 0.522 + 0.296 = 0.818
```

7.2 Behavior Risk

Captures temporal anomalies:

```
age_norm = min(1.0, account_age_days / 365.0)
behavior_risk = 0.55 × (1 − age_norm) + 0.45 × post_hour_cluster_score
```

`(1 − age_norm)` is high for newly created accounts (fakes are created right before the operation starts). `post_hour_cluster_score` measures alignment with the flagged cluster's mean posting hour (see §7.5).

Example: Gang member, `account_age_days=67`, `post_hour_cluster_score=0.91`:

```
age_norm = 67/365 = 0.184
behavior_risk = 0.55×(1−0.184) + 0.45×0.91 = 0.55×0.816 + 0.4095
              = 0.449 + 0.410 = 0.859
```

7.3 Graph Risk

The most predictive signal once the investigation has started:

```
flagged_neighbor_ratio = flagged_neighbor_count /
max(inspected_neighbor_count, 1)
graph_risk = 0.45 × flagged_neighbor_ratio
           + 0.35 × mutual_follow_rate
           + 0.20 × avg_neighbor_photo_reuse
```

`flagged_neighbor_ratio` gets 45% weight — if several of this account's friends are already confirmed fakes, this account is very likely fake too.

Example: After 3 gang members flagged; inspecting a 4th gang member:

```
flagged_neighbor_count = 3 (3 already-flagged accounts in its follow list)
inspected_neighbor_count = 4 (total inspected follows)
mutual_follow_rate = 0.78 (gang mutually follow heavily)
avg_neighbor_photo_reuse = 0.81

flagged_neighbor_ratio = 3/4 = 0.75
graph_risk = 0.45×0.75 + 0.35×0.78 + 0.20×0.81
           = 0.338 + 0.273 + 0.162 = 0.773
```

7.4 Hub Legitimacy

Protects celebrities and legitimate large accounts from false positives:

```
F_MAX = 1,000,000
followers_norm = min(1.0, log(1+follower_count) / log(1+F_MAX))
follow_ratio_norm = min(1.0, (following_count / max(follower_count, 1)) /
5.0)
age_norm = min(1.0, account_age_days / 365.0)

hub_legitimacy = 0.45 × followers_norm
               + 0.25 × (1 − follow_ratio_norm)
               + 0.20 × age_norm
               + 0.10 × (1 − suspicious_mutual_ratio)
```

Four signals of legitimacy:

- Large log-scaled follower count (0.45 weight) — genuine celebrities have millions; fake accounts peak at ~8,000
- Low follow-to-follower ratio (0.25 weight) — celebs follow few, are followed by many; fakes follow aggressively
- Old account (0.20 weight) — real celebrities have accounts years old
- Not mutually following suspicious accounts (0.10 weight) — a celeb being *followed by* fakes doesn't make the celeb fake

Example — Celebrity with 2,000,000 followers:

```
followers_norm = log(2,000,001) / log(1,000,001) = 14.509/13.816 = 1.0
(capped)
follow_ratio_norm = (200 / 2,000,000) / 5.0 = 0.00002 ≈ 0.0
age_norm = min(1.0, 2000/365) = 1.0

hub_legitimacy = 0.45×1.0 + 0.25×(1−0.0) + 0.20×1.0 + 0.10×1.0 = 1.00
```

Example — Gang member:

```
followers_norm = log(3422) / log(1,000,001) = 8.138/13.816 = 0.589
follow_ratio_norm = min(1.0, (847/3422)/5.0) = 0.0495
age_norm = 67/365 = 0.184

hub_legitimacy = 0.45×0.589 + 0.25×(1−0.0495) + 0.20×0.184 + 0.10×0.9
               = 0.265 + 0.238 + 0.037 + 0.090 = 0.630
```

7.5 Post-Hour Cluster Score

Computed dynamically inside `environment.py`, not in `scoring.py`:

```
mean_h = average avg_post_hour across all currently flagged accounts
diff = min(|acc_hour − mean_h|, 24 − |acc_hour − mean_h|)  # wrap-around
post_hour_cluster_score = max(0.0, 1.0 − diff / 6.0)
```

The wrap-around handles the midnight boundary (e.g., 23:00 and 01:00 are 2 hours apart, not 22). A score of 1.0 means posting at exactly the same hour as the flagged cluster. A score of 0.0 means ≥6 hours away.

Why 6.0 as the divisor: 6 hours is a generous "different time zone" threshold. If you post within 6 hours of the gang's schedule, you get partial credit.

Example: Gang posts at mean=14.0. Inspecting an account posting at 14.3:

```
diff = |14.3 − 14.0| = 0.3
post_hour_cluster_score = 1.0 − 0.3/6.0 = 0.950
```

7.6 Composite Fake Risk

```
fake_risk = clip(
    0.30 × node_risk
  + 0.25 × behavior_risk
  + 0.45 × graph_risk
  − 0.25 × hub_legitimacy,
  0.0, 1.0
)
```

Weight rationale:

- **Graph risk 0.45** — structural signals are hardest for fakes to hide. Mutual follow density requires real coordination; once you find one member, the whole cluster lights up.
- **Node risk 0.30** — content signals are strong but can appear on decoys.
- **Behavior risk 0.25** — temporal clustering is a reliable early signal, especially before any flags are set.
- **Hub legitimacy −0.25** — subtractive discount. A celebrity with 5M followers has hub_legitimacy ≈ 1.0, so even if gang members follow them, their risk formula produces: `0.30×0.02 + 0.25×0.05 + 0.45×0.10 − 0.25×1.0 ≈ −0.17 → clipped to 0.0`

Full gang member example (after 3 flags set):

```
node_risk     = 0.818  (photo=0.87, bio=0.74)
behavior_risk = 0.859  (age=67d, cluster_score=0.91)
graph_risk    = 0.773  (ratio=0.75, mutual=0.78, nbr_photo=0.81)
hub_legitimacy= 0.630  (3k followers, 1y old, no celeb)

fake_risk = 0.30×0.818 + 0.25×0.859 + 0.45×0.773 − 0.25×0.630
```

```
           = 0.245 + 0.215 + 0.348 − 0.158
           = 0.650
```

7.7 Risk Classification

```
fake_risk < 0.35    → "normal"
0.35 ≤ risk < 0.60 → "suspect"
risk ≥ 0.60         → "confirmed_fake"   (formula-level; explicit flag
overrides)
```

7.8 Grader Score (Submission Metric)

This normalised [0.0, 1.0] score is returned by the `/grader` endpoint:

```
recall    = tp / 10
precision = tp / max(tp + fp, 1)
efficiency = max(0.0, (max_steps − steps_used) / max_steps)

if recall ≥ 0.8 AND precision ≥ 0.7:
    score = 0.55 + 0.20×recall + 0.15×precision + 0.10×efficiency
else:
    score = 0.30×recall + 0.10×precision
```

Maximum possible score: `0.55 + 0.20×1.0 + 0.15×1.0 + 0.10×1.0 = 1.00` (requires all 10 found, no false positives, and 0 steps used — perfect play)

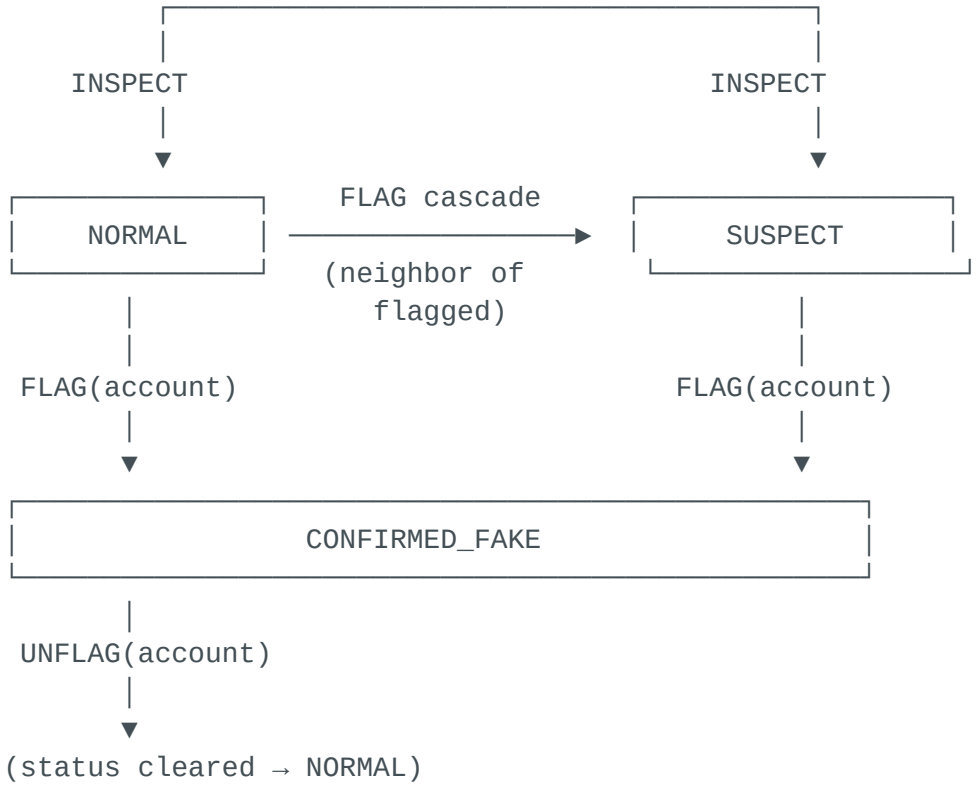
Win threshold score: `0.55 + 0.20×0.8 + 0.15×0.7 + 0.10×0 = 0.55 + 0.16 + 0.105 = 0.815`

Partial credit examples:

- Found 6/10, no false positives: `0.30×0.6 + 0.10×1.0 = 0.18 + 0.10 = 0.28`
- Found 9/10, 3 false positives: recall=0.9, precision=9/12=0.75 → win: `0.55 + 0.18 + 0.113 = 0.843`

---

8. Account Status State Machine



When FLAG(X) is called:

1. X → CONFIRMED_FAKE
2. For every account Y in X's follow list:
   - If Y is visible AND Y is NORMAL: Y → SUSPECT
3. All already-inspected accounts that follow X are re-profiled (their `flagged_neighbor_count` increases, which raises their `fake_risk_score`)

Why SUSPECT matters:

- The `suspect_ids` field in the observation lists all SUSPECT accounts not yet inspected
- Both the rule engine and the LLM treat these as highest priority for the next INSPECT
- This creates an efficient cascade: flag one → inspect suspects → some are gang → flag them → more suspects appear → repeat until cluster is exhausted

Example cascade on easy task:

```
Step 1: INSPECT acc_0003 (gang member) → no flags yet, fake_risk ≈ 0.45
Step 2: FLAG acc_0003
        → acc_0017, acc_0029, acc_0041 become SUSPECT (they follow
acc_0003)
        → obs.suspect_ids = ["acc_0017", "acc_0029", "acc_0041"]
```

```
Step 3: INSPECT acc_0017 (gang member) → fake_risk now 0.72
(flagged_neighbor_count=1)
Step 4: FLAG acc_0017
         → acc_0003 (already flagged), acc_0029, acc_0041, acc_0055 get
SUSPECT
         → acc_0003, acc_0017 profiles refreshed (their mutual flags
increased)
Step 5: INSPECT acc_0029 → fake_risk = 0.81 (flagged_neighbor_count=2)
...
```

Each FLAG makes the next gang member easier to find because their risk score rises.

---

9. The LLM Policy (Qwen3 via Bedrock)

File: `agent/policy.py`

9.1 Model

Qwen3-Next-80B accessed via AWS Bedrock Marketplace:

```
MODEL_ID = "qwen.qwen3-next-80b-a3b"
```

Called via the Bedrock Converse API:

```
client.converse(
    modelId=MODEL_ID,
    messages=[{"role": "user", "content": [{"text": prompt}]}],
    system=[{"text": SYSTEM_PROMPT}],
    inferenceConfig={"maxTokens": 512, "temperature": 0.4}
)
```

Temperature 0.4 is low enough for consistent action format but high enough to avoid degenerate repetition.

9.2 Prompt Construction

Every step, the policy builds a prompt from three components:

```
[reflections from past episodes]       ← grows richer every episode
[best trajectory few-shot example]     ← best win ever, showing the full
action log
── CURRENT CASE ──
[formatted observation]                ← status badges, risk scores, suspect
list
What is your next action?
```

9.3 Observation Formatting

The `_format_observation()` function converts the typed `FakeGangObservation` into a text block. Accounts are **sorted by `fake_risk_score` descending**, with status badges prepended:

```
TASK: EASY | Steps remaining: 22
Currently flagged (3/10): acc_0003, acc_0017, acc_0029
Suspects not yet inspected (4): acc_0041, acc_0055, acc_0062, acc_0078

PROFILED ACCOUNTS (sorted by fake_risk_score — highest first):
  [status | risk | node beh graph hub | photo bio mutual | comment ip_count]
  CONFIRMED_FAKE acc_0029 ◄ FLAGGED: risk=0.821 | node=0.82 beh=0.77
graph=0.86 hub=0.63
  SUSPECT        acc_0041: risk=0.714 | node=0.79 beh=0.81 graph=0.74
hub=0.65 fnbr=3(!)
  SUSPECT        acc_0055: risk=0.681 | node=0.71 beh=0.74 graph=0.69
hub=0.67 fnbr=2(!)
  NORMAL         acc_0022: risk=0.121 | node=0.09 beh=0.31 graph=0.03
hub=0.84 [HUB?]
  ...

KNOWN UNINSPECTED IDs: acc_0062, acc_0078, acc_0091, ...

Environment message: Flagged acc_0029 as suspected fake.
```

Key formatting choices:

- `fnbr=N(!)` highlights when `flagged_neighbor_count > 0` — this is the most actionable graph signal
- `[HUB?]` appears when `hub_legitimacy_score > 0.70` — warns the LLM not to flag
- Status badge width is fixed (13 chars) for visual alignment

9.4 Required Response Format

```
<thinking>
Your reasoning here — which account is most suspicious and why,
what signal you're acting on, what your next move is.
</thinking>
<action>
INSPECT acc_0041
</action>
```

The parser (`_parse_action`) extracts the content inside `<action>...</action>` using regex, then matches to action types. If parsing fails entirely, the fallback inspects the highest-scored uninspected account.

9.5 Retry Logic

```python
for attempt in range(3):
    try:
        raw = invoke_qwen(...)
        action = _parse_action(raw, obs)
        return action, raw
    except Exception as exc:
        wait = 2 ** attempt  # 1s, 2s, 4s
        time.sleep(wait)

# All retries failed → heuristic fallback
return _heuristic_fallback(obs), "[FALLBACK]"
```

---

10. Reflexion — How the Agent Learns

Files: `agent/reflection.py`, `agent/memory.py`

The agent **cannot** update Qwen3's weights — Bedrock is a black-box API. Instead, it learns via **Reflexion**: post-episode lessons are written as text and injected into future prompts.

10.1 Learning Loop

```
Episode N
  1. LLM acts using: system_prompt + reflections[1..4] + best_trajectory
```

```
  2. Episode ends → WIN or LOSS
  3. Post-episode learning:
     If LOSS:
       → generate_reflection(action_log, outcome) → Qwen writes a lesson
       → lesson stored to memory/reflections_easy.jsonl
     If WIN:
       → save trajectory to memory/best_trajectory_easy.json (if better
reward)
       → generate_success_reflection() → Qwen writes what worked
       → stored to reflections

Episode N+1
  → get_reflections("easy", n=4) returns last 4 lessons
  → get_best_trajectory("easy") returns best win as few-shot example
  → both injected into prompt → LLM has learned from its past
```

10.2 Reflection Generation

A separate Qwen3 call is made after each episode with this prompt:

```
CASE DEBRIEF — Episode 12
Task difficulty: MEDIUM
Outcome: FAILURE
Steps used: 50/50
Result: [LOSS] TP=6 FP=3 FN=4 Recall=0.60 Precision=0.67

INVESTIGATION LOG:
  1. INSPECT acc_0022
  2. INSPECT acc_0037
  ...
  20. SUBMIT

Write a 2-3 sentence lesson for your future self based on this case.
```

Example generated reflection:

> "The starting accounts were all real; I wasted 8 steps inspecting low-signal nodes before pivoting. When photo_reuse and bio_template are both below 0.3 after 3 inspections, immediately use INVESTIGATE_NETWORK to jump to a different graph region. Once I found the first gang member at step 14, I should have cascaded faster via SUSPECT accounts rather than continuing to inspect unknown IDs."

This lesson is stored and appears in Episode 13's prompt, causing the agent to pivot earlier and follow the cascade more aggressively.

10.3 Best Trajectory (Few-Shot Example)

The first episode that wins is saved as a few-shot example. Every subsequent win replaces it only if the reward is higher. The trajectory appears in the prompt as:

```
── EXAMPLE SUCCESSFUL CASE (task=easy, reward=+14.20) ──
  1. INSPECT acc_0012
  2. INSPECT acc_0037
  3. FLAG acc_0037
  4. INSPECT acc_0041    (suspect — cascaded from acc_0037)
  5. FLAG acc_0041
  ...
  → [WIN] TP=10 FP=0 FN=0 Recall=1.00 Precision=1.00
```

The LLM sees a concrete example of the exact pattern that leads to a perfect win, and mirrors this strategy.

10.4 Memory Persistence

All memory is stored in `memory/` as flat files:

```
memory/
├── reflections_easy.jsonl      # one JSON entry per reflection
├── reflections_medium.jsonl
├── reflections_hard.jsonl
├── best_trajectory_easy.json   # single best win per task
├── best_trajectory_medium.json
├── best_trajectory_hard.json
├── wins_easy.jsonl             # episode-level win history (for alpha)
├── wins_medium.jsonl
├── wins_hard.jsonl
├── alpha_easy.json             # current α for this task
├── alpha_medium.json
└── alpha_hard.json
```

The `memory/` directory is a Docker volume (`VOLUME ["/app/memory"]`), so all learning persists across container restarts and redeployments.

---

11. Hybrid Policy — The Novel

Contribution

File: `agent/hybrid_policy.py`

The key insight: **a new LLM agent starts dumb but improves over time. A rule engine is always consistent but cannot adapt.** The hybrid policy exploits both: rules provide a safety net early while the LLM builds its track record; once the LLM proves itself, rules step back.

11.1 The Problem with Pure LLM

In the first few episodes:

- No reflections have been generated yet
- No successful trajectory to use as a few-shot example
- The LLM is essentially guessing based only on the system prompt
- Win rate on `easy` episodes ≈ 30% at episode 1 (single-digit recall)

A deterministic rule engine using `fake_risk_score` thresholds would achieve ~60% win rate on `easy` from episode 1, with zero learning overhead.

11.2 The Problem with Pure Rules

Rules use fixed thresholds. They cannot:

- Adapt to the evasion events in hard mode
- Prioritise which SUSPECT to inspect based on context
- Recognise unusual configurations (e.g., decoys clustered near gang members)
- Balance exploration vs. exploitation optimally

The LLM, given enough reflections, learns these nuances.

11.3 Alpha: The Trust Weight

α (alpha) is a per-task value in [0.20, 1.00] representing the agent's current trust in the LLM:

```
α = 0.20 + 0.80 × recent_win_rate × reflection_factor

where:
  recent_win_rate = wins in last 10 episodes for this task
  reflection_factor = min(1.0, n_reflections / 4.0)
```

`reflection_factor` ensures the LLM must accumulate at least **4 reflections** before it can reach full trust — pure win rate is not enough, because the LLM needs to have demonstrably learned from failures.

Alpha trajectory over training:

Episode	Wins (last 10)	Reflections	reflection_factor	α
1	0/0 → wr=0%	0	0.00	0.20
5	1/5 → wr=20%	4	1.00	0.36
10	5/10 → wr=50%	9	1.00	0.60
20	8/10 → wr=80%	19	1.00	0.84
35	9/10 → wr=90%	34	1.00	0.92

α starts at 0.20 (rules dominate) and climbs toward 1.0 as the LLM wins consistently and accumulates lessons.

11.4 Rule Action + Confidence

`get_rule_action(obs)` returns `(FakeGangAction, float)` where the float is the rule's confidence in its own decision:

Situation	Action	Confidence
Steps remaining = 0	SUBMIT	1.00
Uninspected SUSPECT accounts exist	INSPECT suspects[0]	0.95
Inspected account: fake_risk ≥ 0.85	FLAG that account	0.95
Inspected account: fake_risk in [threshold, 0.85)	FLAG that account	0.70 + (risk − threshold) × 0.60

Situation	Action	Confidence
10 accounts already flagged	SUBMIT	0.85
Steps remaining ≤ 3	SUBMIT	0.90
Uninspected accounts available	INSPECT top candidate	0.30
Nothing to do	SUBMIT	0.75

Confidence values are calibrated such that:

- Structural/safety decisions (out of steps, cascade suspects) have confidence ≥ 0.90
- Direct flag decisions have confidence ≥ 0.70
- Exploratory decisions have confidence 0.30 (the rule is just suggesting, not insisting)

11.5 Blending Decision

```
rule_action, rule_conf = get_rule_action(obs)
llm_action, raw_llm   = get_action(obs, reflections, few_shot, temperature)

if rule_action == llm_action:          # same type AND same account_id
    mode = "agree"
    final = llm_action

elif rule_conf >= alpha:               # rule is confident enough to
override
    mode = f"rule_override(c={rule_conf:.2f},α={alpha:.2f})"
    final = rule_action

else:                                  # LLM is trusted; rule doesn't insist
    mode = f"llm(c={rule_conf:.2f}<α={alpha:.2f})"
    final = llm_action
```

Why this works mathematically:

The condition `rule_conf >= alpha` creates a natural threshold system:

- At **α=0.20** (early training, no history):
    - Rules win whenever confidence ≥ 0.20
    - The only exploratory INSPECT (confidence=0.30) still beats α=0.20

    - So rules dominate: ~90% of decisions are rule-driven
    - Effectively acts like the rule-based baseline agent

- At **α=0.50** (moderate trust, mixed results):

    - Rules win when confidence ≥ 0.50
    - Safety decisions (suspects, forced submit) still override: conf=0.95 > 0.50
    - Exploratory decisions (conf=0.30) now go to LLM: 0.30 < 0.50
    - The LLM controls exploration; rules control safety

- At **α=0.84** (high trust, consistent wins):

    - Rules win only when confidence ≥ 0.84
    - Only the two highest-confidence situations still override: forced submit (1.00) and uninspected suspects (0.95)
    - Everything else goes to the LLM, including direct flag decisions

- At **α=1.00** (full trust):

    - Rules never win (confidence is always < 1.00, since 1.00 only fires at steps_remaining=0 which the LLM also handles)
    - Pure LLM mode

11.6 Disagreement Examples

Example 1 — Early training (α=0.25), LLM exploring, rule insisting on suspect:

```
Rule:  INSPECT acc_0041  (SUSPECT account)  confidence=0.95
LLM:   INSPECT acc_0099  (random exploration)
Rule wins: 0.95 ≥ 0.25 → INSPECT acc_0041
mode = "rule_override(c=0.95,α=0.25)"
```

Example 2 — Mid training (α=0.60), LLM flags a high-risk account:

```
Rule:  INSPECT acc_0041  (uninspected suspect)  confidence=0.95
LLM:   FLAG acc_0055  (fake_risk=0.79, already inspected)
Rule wins: 0.95 ≥ 0.60 → INSPECT acc_0041
mode = "rule_override(c=0.95,α=0.60)"
```

(Both actions are useful; the rule correctly prioritises cascade suspects before random flags)

Example 3 — High trust (α=0.85), LLM has learned to prioritise smarter:

```
Rule:  INSPECT acc_0041  (exploratory, conf=0.30)
LLM:   FLAG acc_0055  (fake_risk=0.88, very high confidence)
LLM wins: 0.30 < 0.85 → FLAG acc_0055
mode = "llm(c=0.30<α=0.85)"
```

Example 4 — Both agree (most common case in late training):

```
Rule:  INSPECT acc_0041  (SUSPECT, conf=0.95)
LLM:   INSPECT acc_0041  (LLM also noticed the suspect badge)
mode = "agree"
```

11.7 Alpha Persistence

After every episode, `train.py` does:

```python
# Record outcome
memory.record_win(task, won, episode_num)

# Recompute alpha with updated win history
new_wr = memory.recent_win_rate(task, n=10)
new_alpha = compute_alpha(new_wr, n_reflections)

# Save for next run (even if container restarts)
memory.save_alpha(task, new_alpha)
```

Alpha is stored in `memory/alpha_{task}.json` and loaded at the start of each training run. This means the agent's trust level is preserved across Docker restarts — it doesn't reset to 0.20 every time.

11.8 Mode Logging

Every episode's metrics include a mode breakdown:

```
{
  "alpha_used": 0.42,
  "mode_agree": 11,
  "mode_rule": 7,
  "mode_llm": 4
}
```

The training printer shows this per episode:

```
Ep  12 | easy   | WIN  | reward= +12.40 | recall=1.00 prec=0.91 | steps=21 |
wr=60% | α=0.42 | agree=11 rule=7 llm=4
```

You can watch the transition: early episodes have high `rule` counts; later episodes have high `agree` counts (LLM learned to make the same decisions as the rules, but also brings in strategic reasoning the rules can't).

---

12. Training Loop End-to-End

File: `train.py`

12.1 Curriculum

Phase	Episodes	Task	Goal
1	1–20	easy	Learn basic signal thresholds, build first reflections
2	21–35	medium	Handle decoys, learn evasion response
3	36–50	hard	Feature-only detection, persistent evasion

Seeds rotate deterministically: `seed = (episode_num + task_offset) % 50` so the agent sees all 50 pre-generated episodes before revisiting any.

12.2 Per-Episode Flow

```
for ep in range(n_episodes):
```

```
  1. DETERMINE TASK
     current_task = curriculum_task(ep) or fixed task

  2. COMPUTE ALPHA
     n_refs = memory.reflection_count(current_task)
     wr = memory.recent_win_rate(current_task, n=10)
     alpha = 0.20 + 0.80 × wr × min(1.0, n_refs/4)

  3. LOAD CONTEXT
     reflections = memory.get_reflections(task, n=4)   # last 4 lessons
     few_shot = memory.get_best_trajectory(task)        # best win so far

  4. RUN EPISODE (hybrid policy)
     obs = env.reset(task, seed)
     while not obs.done:
         rule_action, rule_conf = get_rule_action(obs)
         llm_action, raw_llm = get_action(obs, reflections, few_shot, α,
temperature)
         final = blend(rule_action, llm_action, rule_conf, alpha)
         obs = env.step(final)

  5. POST-EPISODE LEARNING
     memory.record_win(task, won, ep)
     new_alpha = compute_alpha(updated_wr, n_refs)
     memory.save_alpha(task, new_alpha)

     if won:
         memory.add_trajectory(task, action_log, final_msg, reward, ep)
         if new_best_or_no_refs:
             reflection = generate_success_reflection(...)
             memory.add_reflection(task, reflection, ep, reward)
     else:
         reflection = generate_reflection(task, action_log, final_msg, ...)
         memory.add_reflection(task, reflection, ep, reward)

  6. LOG
     print per-episode stats: task, win/loss, reward, recall, precision,
                              steps, win_rate, α, mode breakdown
```

12.3 Metrics Saved

Every 5 episodes, metrics are flushed to `runs/metrics.jsonl`:


```
{
  "episode": 15,
  "task": "easy",
  "seed": 14,
  "won": true,
  "reward": 13.20,
  "steps_used": 23,
  "recall": 1.00,
  "precision": 0.91,
```

```
  "action_log": ["INSPECT acc_0022", "INSPECT acc_0037", ...],
  "final_message": "[WIN] TP=10 FP=1 FN=0 ...",
  "n_reflections_used": 4,
  "had_few_shot": true,
  "alpha_used": 0.52,
  "mode_agree": 13,
  "mode_rule": 6,
  "mode_llm": 4,
  "timestamp": "2026-04-01T10:23:41"
}
```

---

13. API Reference

File: `server/app.py`

GET /health

```
{"status": "healthy"}
```

GET /tasks

```
{
  "tasks": ["easy", "medium", "hard"],
  "descriptions": {
    "easy":   "50 accounts, 10 fakes, no evasion, 30 steps",
    "medium": "200 accounts, 10 fakes + 20 decoys, evasion at step 20, 50
steps",
    "hard":   "1000 accounts, 10 fakes + 50 decoys, recurring evasion, 80
steps"
  },
  "action_schema": {
    "action_type": ["inspect", "investigate_network", "flag", "unflag",
"submit"],
    "account_id": "string (required for all actions except submit)"
  },
  "score_range": [0.0, 1.0]
}
```

POST /reset

Request:

```
{"task": "easy", "seed": 0}
```

Response: `StepResponse` with initial observation.

POST /step

Request: Any `FakeGangAction`:

```
{"action_type": "inspect", "account_id": "acc_0042"}
{"action_type": "flag", "account_id": "acc_0017"}
{"action_type": "submit"}
```

Response: `StepResponse` with updated observation, done flag, and reward.

GET /state

Returns current episode metadata:

```
{
  "episode_id": "uuid",
  "step_count": 12,
  "task": "easy",
  "score_so_far": -0.12,
  "evasion_count": 0,
  "network_size": 50,
  "gang_size": 10,
  "episode_seed": 0
}
```

GET /grader

Returns the normalised grader score after SUBMIT. Error 400 if episode not done.

```
{"score": 0.871, "task": "easy", "episode_id": "uuid"}
```

POST /baseline

Runs the rule-based agent on all three tasks (seed=0) and returns scores:

```
{
  "scores": {"easy": 0.871, "medium": 0.743, "hard": 0.612},
  "agent": "rule_based"
}
```

---

14. Docker Deployment

File: `server/Dockerfile`

14.1 Build

```
cd fake_gang_env
docker build -f server/Dockerfile -t fake-gang-env .
```

Build takes ~10 seconds because:

- The `.dockerignore` excludes `episodes/` (109 MB), `memory/`, `runs/`
- Python wheels are pre-downloaded to `wheels/` — no network access during `pip install`
- No `apt-get` installs needed (everything is pure Python)

14.2 Run

```
docker run -it \
  -e AWS_ACCESS_KEY_ID=your_key \
  -e AWS_SECRET_ACCESS_KEY=your_secret \
  -v $(pwd)/memory:/app/memory \
  -v $(pwd)/runs:/app/runs \
  -p 8000:8000 \
  fake-gang-env
```

The volumes preserve all learning between runs. When you restart the container, the agent continues from where it left off (α values, reflections, best trajectories).

14.3 Environment Variables

Variable	Default	Description
`AWS_ACCESS_KEY_ID`	(required)	For Bedrock/Qwen3 access
`AWS_SECRET_ACCESS_KEY`	(required)	For Bedrock/Qwen3 access
`AWS_DEFAULT_REGION`	`us-east-1`	Bedrock region
`TRAIN_TASK`	`` (curriculum)	Fix to `easy`/`medium`/`hard`
`TRAIN_EPISODES`	`50`	Total training episodes
`TRAIN_TEMP`	`0.4`	LLM sampling temperature
`TRAIN_VERBOSE`	`0`	Set `1` for per-step action logging
`SERVER_PORT`	`8000`	FastAPI port

14.4 Startup Sequence (run.sh)

```
1. Validate AWS credentials (exits if missing)
2. python server/generator.py    → generates/overwrites 150 episode JSON
files (~1s)
3. uvicorn server.app:app        → starts the environment server
4. Python urllib health check    → polls /health until ready (no curl
needed)
5. python train.py               → runs the full training loop
```

---

15. Submission Requirements

All three submission requirements are satisfied:

15.1 /tasks with action_schema

The `/tasks` endpoint returns the `action_schema` dict listing all valid `action_type` values and the `account_id` field description. Graders can discover the full action space without reading code.

15.2 /grader

After calling `SUBMIT` (via `/step`), call `GET /grader` to retrieve the normalised [0.0, 1.0] grader score. Returns 400 if the episode is not yet done.

The score formula (see §7.8) rewards recall, precision, and efficiency. Maximum score 1.0 requires finding all 10 gang members with no false positives and using no steps.

15.3 /baseline

`POST /baseline` imports `inference.py`'s `run_rule_based_episode` and runs it on all three tasks with seed=0. Returns:

```
{"scores": {"easy": X, "medium": Y, "hard": Z}, "agent": "rule_based"}
```

15.4 inference.py

Library mode (used by `/baseline`):

```
from inference import run_rule_based_episode
score = run_rule_based_episode(env, task="easy", seed=0)
# Returns float in [0.0, 1.0]
```

CLI mode (connect to running server):

```
python inference.py --url http://localhost:8000
# → {"scores": {"easy": 0.87, "medium": 0.74, "hard": 0.61}, "agent":
"rule_based"}
```

CLI mode (local, no server needed):

```
python inference.py --local
```

The rule-based strategy:

1. If SUSPECT accounts are uninspected → INSPECT highest suspect
2. If any inspected account has `fake_risk_score ≥ threshold` and not flagged → FLAG it
3. If no immediate flag or suspect → INSPECT highest-risk uninspected account
4. If steps ≤ 3 or 10 flags placed → SUBMIT

Thresholds by task: easy=0.60, medium=0.50, hard=0.45.

15.5 validate.py

Runs 24 checks split between local (no server) and HTTP:

```
python validate.py --local            # 9 local checks only
python validate.py --url http://...   # all 24 checks (requires running
server)
```

Checks include:

- scoring.py math correctness (gang risk ≥ 0.60, celebrity risk < 0.20, perfect score = 1.00)
- models.py has all new fields (fake_risk_score, suspect_ids, AccountStatus)
- environment.py SUSPECT cascade triggers after FLAG
- inference.py runs without error and returns [0,1] float
- episodes have new features (comment_repeat_score, shared_ip_count, celeb_ids)
- /health reachable
- /tasks has action_schema and score_range
- /reset works for all three tasks
- /step supports INSPECT, FLAG, SUBMIT
- /grader returns [0,1] float after SUBMIT
- /baseline returns 3 valid scores

---

16. Verification & Validation

Quick smoke test

```
cd fake_gang_env

# Test scoring math
python3 -c "
import sys; sys.path.insert(0,'server')
from scoring import compute_fake_risk, compute_hub_legitimacy, grader_score

gang_r = compute_fake_risk(0.75, 0.65, 0.85, 0.10)
hub    = compute_hub_legitimacy(2_000_000, 200, 2000, 0.05)
celeb  = compute_fake_risk(0.02, 0.02, 0.10, hub)
assert gang_r >= 0.60,  f'Gang risk too low: {gang_r}'
assert celeb  <  0.20,  f'Celebrity risk too high: {celeb}'
assert grader_score(10, 0, 0, 0, 30) == 1.0
print(f'Gang risk={gang_r}  Celeb risk={celeb}  Perfect score=1.0  OK')
"

# Test hybrid policy + cascade
python3 -c "
import sys, json; sys.path.insert(0,'server')
from models import FakeGangAction, ActionType
from environment import FakeGangEnvironment
from agent.hybrid_policy import get_rule_action, compute_alpha

env = FakeGangEnvironment()
obs = env.reset(task='easy', seed=0)
gang = json.loads(open('episodes/easy_000.json').read())['gang_member_ids']
obs = env.step(FakeGangAction(action_type=ActionType.INSPECT,
account_id=gang[0]))
obs = env.step(FakeGangAction(action_type=ActionType.FLAG,
account_id=gang[0]))
assert len(obs.suspect_ids) > 0, 'Cascade failed'
action, conf = get_rule_action(obs)
assert action.account_id in obs.suspect_ids, 'Rule not prioritising
suspects'
print(f'Cascade OK: {len(obs.suspect_ids)} suspects. Rule → INSPECT
{action.account_id} (conf={conf:.2f})')
a0, a1, a2 = compute_alpha(0,0), compute_alpha(0.5,2), compute_alpha(1,4)
print(f'Alpha: min={a0} mid={a1} max={a2}')
"

# Full local validate
python3 validate.py --local
```

Full HTTP validation (requires running server)

```
python3 -m uvicorn server.app:app --port 8001 &
sleep 3
python3 validate.py --url http://localhost:8001
```

Expected output: `Results: 24/24 passed — all OK`